UNIVERSIDADE FEDERAL DO PARANÁ
Bacharelado em Ciência da Computação

LAISA MARCELINO SANTOS RODRIGUES
GRR20243783 – `lmsr24`

TRABALHO 01 - MODELAGEM EM PROGRAMAÇÃO LINEAR

Curitiba – PR
2026

Sumário

1 Introdução **2**

2 Descrição do problema **2**

3 Modelagem matemática **3**
3.1 Variáveis de decisão 3
3.2 Função objetivo 3
3.3 Restrições de atendimento das necessidades mínimas 3
3.4 Restrições de limite máximo 3
3.5 Modelo final 4
3.6 Justificativa da modelagem 4

4 Implementação **4**
4.1 Visão geral 4
4.2 Leitura da instância 5
4.3 Geração do modelo LP 5
4.4 Compilação e execução 5

5 Possibilidade de inviabilidade **5**

6 Exemplos **6**
6.1 Exemplo do enunciado 6
6.2 Exemplo viável simples 6
6.3 Exemplo inviável 7

7 Conclusão **7**

1 Introdução

Este trabalho apresenta a modelagem e a implementação, por programação linear, de uma solução para o problema *Lista de remédios*. O objetivo é determinar quantos comprimidos de cada tipo devem ser ingeridos por dia, de modo a satisfazer as quantidades mínimas exigidas de determinados componentes, respeitar limites máximos de consumo para componentes restritos e minimizar o custo total diário.

A metodologia do trabalho parte do princípio de que um problema prático deve primeiro ser convertido em uma formulação matemática, para então permitir o desenvolvimento de uma solução computacional adequada. Como observam Kleinberg e Tardos (2006), parte central do trabalho algorítmico consiste justamente em identificar, em meio aos detalhes de uma aplicação, um núcleo matemático limpo e tratável.

No contexto da programação linear, essa formulação corresponde à escolha de variáveis de decisão, de uma função objetivo linear e de restrições lineares que expressem a descrição do problema. Conforme apresentam Matoušek e Gärtner (2007), um programa linear busca otimizar uma função linear sujeita a um conjunto de equações e desigualdades lineares, podendo ser escrito, em forma padrão, como a otimização de $c^T x$ sujeita a $Ax \leq b$.

Conforme especificado no enunciado, este trabalho não implementa o método simplex. Desse modo, o principal objetivo desta tarefa consiste em gerar, na saída padrão, um arquivo no formato aceito pelo resolvedor *lp_solve*. Assim, o programa desenvolvido lê uma instância pela entrada padrão e produz um modelo linear inteiro equivalente ao problema original, visto que, para o problema da *Lista de remédios* faz sentido que os comprimentos sejam valores inteiros.

2 Descrição do problema

Considere os seguintes dados de entrada:

- c: número total de componentes;
- p: número de tipos de comprimidos;
- q_j: quantidade mínima diária necessária do componente j, para $j = 1, 2, \ldots, c$;
- $r_{i,j}$: quantidade do componente j presente em um comprimido do tipo i, para $i = 1, 2, \ldots, p$;
- v_i: preço unitário do comprimido do tipo i;
- k: número de componentes com consumo limitado;
- f_t: índice do componente restrito associado à restrição t, para $t = 1, 2, \ldots, k$;
- l_t: limite máximo permitido para o componente f_t.

O problema consiste em determinar quantos comprimidos de cada tipo devem ser tomados por dia. Como comprimidos são unidades discretas, isto é, são quantidades que só podem assumir valores inteiros e separados, sem possibilidade de divisão contínua, a quantidade escolhida de cada tipo deve ser inteira e não negativa.

3 Modelagem matemática

3.1 Variáveis de decisão

Para cada tipo de comprimido i, com $i = 1, 2, \ldots, p$, define-se a variável

$$x_i = \text{quantidade de comprimidos do tipo } i \text{ ingeridos por dia.}$$

Como não faz sentido consumir quantidade negativa de comprimidos e como a unidade de decisão é discreta, impõe-se

$$x_i \in \mathbb{Z}_{\geq 0}, \qquad i = 1, 2, \ldots, p.$$

3.2 Função objetivo

Cada comprimido do tipo i possui custo unitário v_i. Logo, o custo total diário associado a uma escolha $(x_1, x_2, \ldots, x_p)$ é

$$\sum_{i=1}^{p} v_i x_i.$$

Como se deseja minimizar o gasto diário com os comprimidos, a função objetivo é

$$\min \sum_{i=1}^{p} v_i x_i.$$

3.3 Restrições de atendimento das necessidades mínimas

Para cada componente j, a soma das quantidades fornecidas por todos os comprimidos consumidos deve ser pelo menos igual à quantidade mínima necessária q_j. Como cada comprimido do tipo i fornece $r_{i,j}$ unidades do componente j, obtém-se, para todo $j = 1, 2, \ldots, c$,

$$\sum_{i=1}^{p} r_{i,j} x_i \geq q_j.$$

Essas restrições garantem que a solução escolhida atenda às necessidades mínimas diárias de cada componente exigido.

3.4 Restrições de limite máximo

Alguns componentes aparecem na lista de restrições por não poderem ultrapassar determinado limite diário. Se o componente f_t possui limite máximo l_t, então a quantidade total ingerida desse componente deve satisfazer, para todo $t = 1, 2, \ldots, k$,

$$\sum_{i=1}^{p} r_{i,f_t} x_i \leq l_t.$$

Essas restrições modelam situações em que substâncias presentes nos comprimidos, embora façam parte da composição, devem ter consumo controlado.

3.5 Modelo final

Seguindo a formulação clássica de programação linear descrita por Matoušek e Gärtner (2007), o modelo completo pode ser escrito como

$$\min \sum_{i=1}^{p} v_i x_i$$

sujeito a

$$\sum_{i=1}^{p} r_{i,j} x_i \geq q_j, \qquad j = 1, 2, \ldots, c,$$

$$\sum_{i=1}^{p} r_{i,f_t} x_i \leq l_t, \qquad t = 1, 2, \ldots, k,$$

$$x_i \in \mathbb{Z}_{\geq 0}, \qquad i = 1, 2, \ldots, p.$$

3.6 Justificativa da modelagem

A modelagem apresentada resolve o problema porque cada variável x_i representa uma decisão concreta sobre quantos comprimidos do tipo i devem ser ingeridos por dia. A função objetivo mede o custo total dessa escolha. As restrições do primeiro grupo garantem que todas as quantidades mínimas exigidas sejam atendidas, enquanto as restrições do segundo grupo impedem que componentes limitados ultrapassem os valores permitidos. Por fim, a modelagem das restrições e a não negatividade das variáveis asseguram que a solução encontrada seja interpretável no contexto do problema.

4 Implementação

4.1 Visão geral

A implementação foi desenvolvida em C com o objetivo de ler uma instância do problema pela entrada padrão e escrever, na saída padrão, o modelo equivalente no formato aceito pelo *lp_solve*. O programa não executa a otimização; ele apenas constrói a formulação do problema que posteriormente pode ser processada pelo resolvedor.

A estrutura do projeto é contém os seguintes arquivos principais:

- `main.c`: ponto de entrada do programa;
- `problem.h`: definição das estruturas de dados e dos protótipos das funções;
- `problem.c`: leitura da instância, geração do modelo LP e liberação da memória;
- `makefile`: compilação do projeto e geração do executável `comprimidos`.

4.2 Leitura da instância

A entrada é composta apenas por números inteiros e segue a ordem definida no escopo do trabalho:

1. leitura de c e p;
2. leitura do vetor q com as quantidades mínimas diárias;
3. leitura das p linhas que descrevem a composição e o preço de cada comprimido;
4. leitura de k;
5. leitura dos pares (f_t, l_t) que descrevem os componentes limitados.

4.3 Geração do modelo LP

A saída gerada segue a sintaxe do *lp_solve*. O formato possui a função objetivo, as restrições e as declarações de variáveis. Como as variáveis do problema são inteiras, a implementação gera também a declaração `int`. Além disso, os nomes das variáveis foram definidos como `x1`, `x2`, ..., `xp`.

Para tornar a saída mais limpa, o programa omite termos com coeficiente zero ao imprimir as expressões lineares.

4.4 Compilação e execução

O projeto possui um `makefile` com alvo padrão `make`, que gera o executável `comprimidos` no diretório corrente, conforme exigido no enunciado.

A compilação pode ser feita com:

```
make
```

A execução típica do programa é feita por redirecionamento da entrada padrão:

```
./comprimidos < exemplos/exemplo_enunciado.in
```

Nesse uso, o arquivo de entrada contém a instância do problema, e a saída produzida é o modelo LP correspondente.

5 Possibilidade de inviabilidade

Como observam Matoušek e Gärtner (2007), um modelo de programação linear pode ser inviável quando não existe vetor que satisfaça simultaneamente todas as restrições. No contexto deste trabalho, isso significa que podem existir instâncias para as quais nenhuma combinação de comprimidos seja capaz de atender, ao mesmo tempo, as quantidades mínimas e os limites máximos exigidos.

Isso pode ocorrer, por exemplo, quando um componente necessário não aparece em quantidade suficiente em nenhum conjunto de comprimidos disponível, ou quando atender à exigência mínima de um componente força a violação do limite máximo de outro componente restrito. Também pode haver inviabilidade em situações estruturalmente contraditórias, como exigir uma quantidade mínima positiva de um componente e, ao mesmo tempo, impor a esse mesmo componente um limite máximo inferior ao valor requerido.

A implementação desenvolvida não tenta detectar inviabilidade diretamente. O programa apenas gera o modelo LP correspondente à instância lida. A verificação de existência ou não de solução viável é de responsabilidade do *lp_solve*.

6 Exemplos

6.1 Exemplo do enunciado

O exemplo apresentado no enunciado é dado por:

```
3 2
10 5 0
5 1 0 2
2 4 1 3
1
3 0
```

Para essa entrada, uma saída possível do programa é:

```
min: 2 x1 + 3 x2;

5 x1 + 2 x2 >= 10;
1 x1 + 4 x2 >= 5;
1 x2 >= 0;

1 x2 <= 0;

x1 >= 0;
x2 >= 0;

int x1, x2;
```

Observe que a terceira restrição de quantidade mínima e a restrição de limite foram simplificadas pela omissão de coeficientes nulos. Assim, a expressão geral do componente 3,

$$0x1 + 1x2,$$

foi impressa apenas como `1 x2`, que é equivalente.

6.2 Exemplo viável simples

Foi incluído também o seguinte exemplo adicional viável:

```
2 3
8 6
2 1 3
1 3 4
4 0 5
1
2 6
```

Esse caso é útil para testar uma instância em que as exigências mínimas e os limites máximos podem ser satisfeitos simultaneamente.

6.3 Exemplo inviável

Outro exemplo é:

```
2 2
5 1
5 0 2
0 1 1
1
1 4
```

Nesse caso, o componente 1 precisa atingir pelo menos 5 unidades por dia, mas ao mesmo tempo a restrição limitada impõe que ele não ultrapasse 4 unidades. Logo, a instância é inviável.

7 Conclusão

Neste trabalho foi apresentada uma modelagem por programação linear inteira para o problema *Lista de remédios*, bem como uma implementação em C responsável por traduzir instâncias do problema para o formato aceito pelo *lp_solve*.

Referências

- ANDRÉ GUEDES; MURILO V. G. DA SILVA. *Primeiro Trabalho Prático*. CI1238 - Otimização / INFO7070 - Tópicos Especiais I, 31 mar. 2026.
- KLEINBERG, Jon; TARDOS, Éva. *Algorithm Design*. 1. ed. Boston: Pearson Education, 2006.
- MATOUŠEK, Jiří; GÄRTNER, Bernd. *Understanding and Using Linear Programming*. Berlin; Heidelberg: Springer-Verlag, 2007.
- *lp_solve 5.5 – LP file format*. Disponível em: `http://lpsolve.sourceforge.net/5.5/lp-format.htm`. Acesso em: 19 abr. 2026.